

Capítulo 16

Introdução a Classes e Objetos

Fazendo a Diferença — Resolução Didática

“Os exercícios Fazendo a Diferença do Capítulo 16 nos pedem para aplicar nosso conhecimento recém-adquirido de classes e objetos a problemas reais de saúde pública: cálculo da frequência cardíaca durante exercícios e perfil de saúde eletrônico de uma pessoa. Esses exercícios consolidam, em escala mais ambiciosa, todos os conceitos de OO do capítulo — e vão além, exigindo cálculos derivados de atributos (idade, IMC, frequência cardíaca máxima) que demonstram o poder da abordagem orientada a objetos.”

Fazendo a Diferença

Sobre estes exercícios. O 16.16 é baseado em uma fórmula da American Heart Association: *frequência cardíaca máxima* = 220 – idade (em anos); *faixa ideal durante exercícios* = entre 50% e 85% da máxima. O 16.17 estende essa abordagem combinando o cálculo da frequência cardíaca com o do índice de massa corporal (IMC), já apresentado no Capítulo 2. Atendendo à recomendação explícita do livro: **essas fórmulas são estimativas estatísticas; um médico ou profissional de saúde qualificado deve ser sempre consultado antes de iniciar ou modificar um programa de exercícios.**

1 Exercício 16.16 — Classe `FrequenciaCardiaca`

Enunciado. Crie a classe `FrequenciaCardiaca` com atributos: primeiro nome, sobrenome, e data de nascimento (atributos separados para dia, mês e ano). Construtor recebe todos esses dados. *Set/get* para cada. Inclua: `obterIdade` (calcula e retorna idade em anos — pede a data atual ao usuário, pois ainda não sabemos obtê-la do sistema); `obterFrequenciaMaxima` (220 – idade); `obterFrequenciaIdeal` (50%–85% da máxima). Aplicação deve pedir os dados, instanciar o objeto e imprimir tudo.

Análise do problema

A classe `FrequenciaCardiaca` é nosso primeiro exemplo de classe com *computações derivadas*: ela não apenas guarda atributos, mas também calcula valores a partir deles. O ponto sutil é *como* a função `obterIdade` obtém a data atual: como ainda não aprendemos a usar bibliotecas de data/hora do sistema (`<ctime>` ou `<chrono>`), o próprio livro orienta-nos a *perguntar ao usuário*.

A fórmula da idade tem uma sutileza: se a pessoa ainda *não* fez aniversário neste ano (mês atual menor que mês de nascimento, ou mês igual mas dia ainda anterior), então a idade é ano atual – ano de nascimento – 1, e não ano atual – ano de nascimento.

Cabeçalho FrequenciaCardiaca.h

```

1  // Exercício 16.16 - FrequenciaCardiaca.h (Fazendo a Diferença)
2  #ifndef FREQUENCIA_CARDIACA_H
3  #define FREQUENCIA_CARDIACA_H
4
5  #include <string>
6
7  class FrequenciaCardiaca {
8  public:
9      FrequenciaCardiaca(std::string pnome, std::string snome,
10                          int diaNasc, int mesNasc, int anoNasc);
11
12      // Set/get para nome
13      void setPrimeiroNome(std::string n);
14      std::string getPrimeiroNome() const;
15
16      void setSobrenome(std::string s);
17      std::string getSobrenome() const;
18
19      // Set/get para data de nascimento
20      void setDiaNascimento(int d);
21      int getDiaNascimento() const;
22
23      void setMesNascimento(int m);
24      int getMesNascimento() const;
25
26      void setAnoNascimento(int a);
27      int getAnoNascimento() const;
28
29      // Calcula idade pedindo data atual ao usuário
30      int obterIdade() const;
31
32      // Calcula frequência cardíaca máxima (220 - idade)
33      int obterFrequenciaMaxima() const;
34
35      // Imprime intervalo da frequência ideal (50% a 85% da máxima)
36      void obterFrequenciaIdeal() const;
37
38  private:
39      std::string primeiroNome;
40      std::string sobrenome;
41      int diaNascimento;
42      int mesNascimento;
43      int anoNascimento;
44  };
45
46  #endif

```

Implementação FrequenciaCardiaca.cpp

```

1  // Exercício 16.16 - FrequenciaCardiaca.cpp
2  #include <iostream>
3  #include "FrequenciaCardiaca.h"
4  using namespace std;
5
6  FrequenciaCardiaca::FrequenciaCardiaca(string pn, string sn,
7                                          int dn, int mn, int an)
8      : primeiroNome(pn), sobrenome(sn),
9        diaNascimento(dn), mesNascimento(mn), anoNascimento(an) {
10 }
11

```

```

12 void FrequenciaCardiaca::setPrimeiroNome(string n) { primeiroNome = n; }
13 string FrequenciaCardiaca::getPrimeiroNome() const { return primeiroNome; }
14
15 void FrequenciaCardiaca::setSobrenome(string s) { sobrenome = s; }
16 string FrequenciaCardiaca::getSobrenome() const { return sobrenome; }
17
18 void FrequenciaCardiaca::setDiaNascimento(int d) { diaNascimento = d; }
19 int FrequenciaCardiaca::getDiaNascimento() const { return diaNascimento; }
20
21 void FrequenciaCardiaca::setMesNascimento(int m) { mesNascimento = m; }
22 int FrequenciaCardiaca::getMesNascimento() const { return mesNascimento; }
23
24 void FrequenciaCardiaca::setAnoNascimento(int a) { anoNascimento = a; }
25 int FrequenciaCardiaca::getAnoNascimento() const { return anoNascimento; }
26
27 int FrequenciaCardiaca::obterIdade() const {
28     int diaAtual, mesAtual, anoAtual;
29
30     cout << "Digite a data atual (dia mes ano): ";
31     cin >> diaAtual >> mesAtual >> anoAtual;
32
33     int idade = anoAtual - anoNascimento;
34
35     // Ajusta caso ainda nao tenha feito aniversario neste ano
36     if (mesAtual < mesNascimento ||
37         (mesAtual == mesNascimento && diaAtual < diaNascimento)) {
38         --idade;
39     }
40     return idade;
41 }
42
43 int FrequenciaCardiaca::obterFrequenciaMaxima() const {
44     return 220 - obterIdade();
45 }
46
47 void FrequenciaCardiaca::obterFrequenciaIdeal() const {
48     int maxima = obterFrequenciaMaxima();
49     int idealMin = maxima * 50 / 100;
50     int idealMax = maxima * 85 / 100;
51
52     cout << "Frequencia cardiaca maxima: " << maxima << " bpm" << endl;
53     cout << "Faixa ideal (50%-85% da maxima): "
54         << idealMin << " - " << idealMax << " bpm" << endl;
55 }

```

Programa de teste main_FrequenciaCardiaca.cpp

```

1 // Exercício 16.16 - main_FrequenciaCardiaca.cpp
2 #include <iostream>
3 #include "FrequenciaCardiaca.h"
4 using namespace std;
5
6 int main() {
7     string pnome, snome;
8     int dia, mes, ano;
9
10    cout << "=== Calculadora de Frequencia Cardiaca ===" << endl;
11    cout << "Primeiro nome: ";
12    cin >> pnome;
13    cout << "Sobrenome: ";
14    cin >> snome;
15    cout << "Data de nascimento (dia mes ano): ";

```

```

16     cin  >> dia >> mes >> ano;
17
18     FrequenciaCardiaca pessoa(pnome, snome, dia, mes, ano);
19
20     cout << "\n--- Dados da pessoa ---" << endl;
21     cout << "Nome: " << pessoa.getPrimeiroNome()
22          << " " << pessoa.getSobrenome() << endl;
23     cout << "Data de nascimento: "
24          << pessoa.getDiaNascimento() << "/"
25          << pessoa.getMesNascimento() << "/"
26          << pessoa.getAnoNascimento() << endl;
27
28     cout << "\n--- Frequencia ideal ---" << endl;
29     pessoa.obterFrequenciaIdeal();
30
31     return 0;
32 }

```

Execução

Saída da execução

```

$ ./test_FreqCardiaca
=== Calculadora de Frequencia Cardiaca ===
Primeiro nome: Maria
Sobrenome: Silva
Data de nascimento (dia mes ano): 15 5 1990

--- Dados da pessoa ---
Nome: Maria Silva
Data de nascimento: 15/5/1990

--- Frequencia ideal ---
Digite a data atual (dia mes ano): 15 5 2026
Frequencia cardiaca maxima: 184 bpm
Faixa ideal (50%-85% da maxima): 92 - 156 bpm

```

Discussão

- Conferência do cálculo: nascida em 15/5/1990, hoje é 15/5/2026. A pessoa *exatamente hoje* completa 36 anos. Logo, $FCM = 220 - 36 = 184$ bpm. Faixa ideal: $184 \times 0,50 = 92$ e $184 \times 0,85 = 156,4 \approx 156$. Tudo correto.
- Observe que `obterIdade` é declarada `const`: ela não modifica os dados-membro do objeto. Mesmo que internamente faça `cin >> ...`, esse `cin` *não* muda o objeto — só lê dados de fora.
- A função `obterFrequenciaIdeal` é `void` porque, em vez de retornar valores, ela *imprime* os dois limites. Alternativamente, poderíamos retornar um `pair<int,int>` ou usar parâmetros de referência de saída — escolhas mais sofisticadas que virão em capítulos futuros.

Erro Comum de Programação

Observe que, na minha implementação didática, `obterFrequenciaIdeal` chama `obterFrequenciaMaxima`, que por sua vez chama `obterIdade` — e `obterIdade` pede a data atual ao usuário. Cada nível da pilha de chamada faz nova solicitação! Em código de produção, deveríamos receber a data atual uma só vez no programa principal e passa-la como parâmetro a essas funções, ou armazenar a idade calculada em cache. Tratamos do tema de

efeitos colaterais em funções-membro a partir do Cap. 18.

Observação de Engenharia de Software

Em um ambiente profissional, o cálculo da idade seria feito comparando a data de nascimento com a data atual obtida do sistema operacional, não da entrada do usuário. C++ moderno oferece `<chrono>` com tipos como `system_clock::now()` que dispensam interação humana. O exercício adota a interatividade por razões didáticas próprias do estágio do livro.

2 Exercício 16.17 — Classe PerfilSaude

Enunciado. Projete uma classe `PerfilSaude` para uma pessoa, com atributos: nome, sobrenome, gênero, data de nascimento (atributos separados), altura (cm) e peso (kg). Construtor recebe todos. *Set/get* para cada. Inclua funções que calculem e retornem: idade em anos, frequências cardíacas máxima e ideal (do 16.16), e o IMC (do 2.32). Aplicação deve solicitar a informação, instanciar o objeto, imprimir o perfil completo (incluindo IMC, frequência cardíaca máxima e faixa ideal), e também mostrar o gráfico de valores de IMC.

Análise do problema

Este exercício amplia o anterior: agora a classe combina *dois* cálculos derivados (frequência cardíaca e IMC). A fórmula do IMC é a vista no Capítulo 2:

$$\text{IMC} = \frac{\text{peso (kg)}}{[\text{altura (m)}]^2}$$

Atenção: a altura está em *centímetros*, então precisamos dividir por 100 antes de elevar ao quadrado. A inversão da ordem das operações (elevar ao quadrado e *depois* converter) seria possível, mas a clareza didática prefere a conversão explícita primeiro.

Cabeçalho PerfilSaude.h

```

1 // Exercício 16.17 - PerfilSaude.h (Fazendo a Diferença)
2 // Perfil de saude com nome, sexo, data de nascimento, altura e peso
3 #ifndef PERFIL_SAUDE_H
4 #define PERFIL_SAUDE_H
5
6 #include <string>
7
8 class PerfilSaude {
9 public:
10     PerfilSaude(std::string pnome, std::string snome,
11                 std::string sexo,
12                 int diaNasc, int mesNasc, int anoNasc,
13                 int alturaCm, int pesoKg);
14
15     // Setters / getters
16     void setPrimeiroNome(std::string n);
17     std::string getPrimeiroNome() const;
18
19     void setSobrenome(std::string s);
20     std::string getSobrenome() const;
21
22     void setSexo(std::string s);
23     std::string getSexo() const;
24
25     void setDiaNascimento(int d);
26     int getDiaNascimento() const;
27
28     void setMesNascimento(int m);
29     int getMesNascimento() const;
30
31     void setAnoNascimento(int a);
32     int getAnoNascimento() const;
33
34     void setAltura(int cm);
35     int getAltura() const;
36

```

```

37     void setPeso(int kg);
38     int  getPeso() const;
39
40     // Calculos
41     int  obterIdade() const;           // pede data atual
42     int  obterFrequenciaMaxima() const; // 220 - idade
43     void obterFrequenciaIdeal() const;  // 50-85% da maxima
44     double obterIMC() const;           // peso(kg) / altura(m)^2
45     void  exibirTabelaIMC() const;     // grafico de faixas
46
47 private:
48     std::string primeiroNome;
49     std::string sobrenome;
50     std::string sexo;
51     int diaNascimento;
52     int mesNascimento;
53     int anoNascimento;
54     int alturaCm;      // em centimetros
55     int pesoKg;        // em quilogramas
56 };
57
58 #endif

```

Implementação PerfilSaude.cpp

```

1  // Exercício 16.17 - PerfilSaude.cpp
2  #include <iostream>
3  #include "PerfilSaude.h"
4  using namespace std;
5
6  PerfilSaude::PerfilSaude(string pn, string sn, string sx,
7                          int dn, int mn, int an,
8                          int alt, int peso)
9      : primeiroNome(pn), sobrenome(sn), sexo(sx),
10      diaNascimento(dn), mesNascimento(mn), anoNascimento(an),
11      alturaCm(alt), pesoKg(peso) {
12  }
13
14  void PerfilSaude::setPrimeiroNome(string n) { primeiroNome = n; }
15  string PerfilSaude::getPrimeiroNome() const { return primeiroNome; }
16
17  void PerfilSaude::setSobrenome(string s) { sobrenome = s; }
18  string PerfilSaude::getSobrenome() const { return sobrenome; }
19
20  void PerfilSaude::setSexo(string s) { sexo = s; }
21  string PerfilSaude::getSexo() const { return sexo; }
22
23  void PerfilSaude::setDiaNascimento(int d) { diaNascimento = d; }
24  int  PerfilSaude::getDiaNascimento() const { return diaNascimento; }
25
26  void PerfilSaude::setMesNascimento(int m) { mesNascimento = m; }
27  int  PerfilSaude::getMesNascimento() const { return mesNascimento; }
28
29  void PerfilSaude::setAnoNascimento(int a) { anoNascimento = a; }
30  int  PerfilSaude::getAnoNascimento() const { return anoNascimento; }
31
32  void PerfilSaude::setAltura(int cm) { alturaCm = cm; }
33  int  PerfilSaude::getAltura() const { return alturaCm; }
34
35  void PerfilSaude::setPeso(int kg) { pesoKg = kg; }
36  int  PerfilSaude::getPeso() const { return pesoKg; }
37

```

```

38 int PerfilSaude::obterIdade() const {
39     int diaAtual, mesAtual, anoAtual;
40     cout << "Digite a data atual (dia mes ano): ";
41     cin >> diaAtual >> mesAtual >> anoAtual;
42
43     int idade = anoAtual - anoNascimento;
44     if (mesAtual < mesNascimento ||
45         (mesAtual == mesNascimento && diaAtual < diaNascimento)) {
46         --idade;
47     }
48     return idade;
49 }
50
51 int PerfilSaude::obterFrequenciaMaxima() const {
52     return 220 - obterIdade();
53 }
54
55 void PerfilSaude::obterFrequenciaIdeal() const {
56     int maxima = obterFrequenciaMaxima();
57     int idealMin = maxima * 50 / 100;
58     int idealMax = maxima * 85 / 100;
59     cout << "Frequencia cardiaca maxima: " << maxima << " bpm" << endl;
60     cout << "Faixa ideal (50%-85% da maxima): "
61         << idealMin << " - " << idealMax << " bpm" << endl;
62 }
63
64 double PerfilSaude::obterIMC() const {
65     double alturaMetros = alturaCm / 100.0;
66     return pesoKg / (alturaMetros * alturaMetros);
67 }
68
69 void PerfilSaude::exibirTabelaIMC() const {
70     cout << "\n--- Tabela de classificacao do IMC (OMS) ---" << endl;
71     cout << "    Abaixo de 18.5    : Abaixo do peso" << endl;
72     cout << "    18.5 a 24.9        : Peso normal" << endl;
73     cout << "    25.0 a 29.9        : Sobrepeso" << endl;
74     cout << "    30.0 a 34.9        : Obesidade grau I" << endl;
75     cout << "    35.0 a 39.9        : Obesidade grau II" << endl;
76     cout << "    40.0 ou mais       : Obesidade grau III" << endl;
77 }

```

Programa de teste main_PerfilSaude.cpp

```

1  // Exercício 16.17 - main_PerfilSaude.cpp
2  #include <iostream>
3  #include <iomanip>
4  #include "PerfilSaude.h"
5  using namespace std;
6
7  int main() {
8      string pnome, snome, sx;
9      int dia, mes, ano, alt, peso;
10
11     cout << "=== Perfil de Saude ===" << endl;
12     cout << "Primeiro nome: "; cin >> pnome;
13     cout << "Sobrenome: "; cin >> snome;
14     cout << "Sexo (M/F): "; cin >> sx;
15     cout << "Data de nascimento (dia mes ano): ";
16     cin >> dia >> mes >> ano;
17     cout << "Altura (em cm): "; cin >> alt;
18     cout << "Peso (em kg): "; cin >> peso;
19

```



```

20     PerfilSaude pessoa(pnome, snome, sx, dia, mes, ano, alt, peso);
21
22     cout << "\n--- Dados Pessoais ---" << endl;
23     cout << "Nome: " << pessoa.getPrimeiroNome()
24           << " " << pessoa.getSobrenome() << endl;
25     cout << "Sexo: " << pessoa.getSexo() << endl;
26     cout << "Data de nascimento: "
27           << pessoa.getDiaNascimento() << "/"
28           << pessoa.getMesNascimento() << "/"
29           << pessoa.getAnoNascimento() << endl;
30     cout << "Altura: " << pessoa.getAltura() << " cm" << endl;
31     cout << "Peso: " << pessoa.getPeso() << " kg" << endl;
32
33     cout << "\n--- IMC ---" << endl;
34     cout << fixed << setprecision(2);
35     cout << "IMC = " << pessoa.obterIMC() << " kg/m^2" << endl;
36     pessoa.exibirTabelaIMC();
37
38     cout << "\n--- Frequencia Cardiaca ---" << endl;
39     pessoa.obterFrequenciaIdeal();
40
41     return 0;
42 }

```

Execução

Saída da execução

```

$ ./test_PerfilSaude
=== Perfil de Saude ===
Primeiro nome: Joao
Sobrenome: Silva
Sexo (M/F): M
Data de nascimento (dia mes ano): 10 3 1985
Altura (em cm): 175
Peso (em kg): 80

--- Dados Pessoais ---
Nome: Joao Silva
Sexo: M
Data de nascimento: 10/3/1985
Altura: 175 cm
Peso: 80 kg

--- IMC ---
IMC = 26.12 kg/m^2

--- Tabela de classificacao do IMC (OMS) ---
Abaixo de 18.5 : Abaixo do peso
18.5 a 24.9 : Peso normal
25.0 a 29.9 : Sobrepeso
30.0 a 34.9 : Obesidade grau I
35.0 a 39.9 : Obesidade grau II
40.0 ou mais : Obesidade grau III

--- Frequencia Cardiaca ---
Digite a data atual (dia mes ano): 15 5 2026
Frequencia cardiaca maxima: 179 bpm
Faixa ideal (50%-85% da maxima): 89 - 152 bpm

```

Discussão

- Conferência do IMC: altura 175 cm = 1,75 m; peso 80 kg. $IMC = 80 / (1,75 \times 1,75) = 80 / 3,0625 \approx 26,12$. Confere. Pelo gráfico da OMS, está na faixa 25,0–29,9: *sobrepeso*.
- Conferência da idade: nascido em 10/3/1985, hoje é 15/5/2026. Mês atual (5) maior que mês de nascimento (3), portanto já fez aniversário neste ano. Idade = 2026 – 1985 = 41 anos. FCM = 220 – 41 = 179. Faixa ideal: 89 a 152. Tudo confere.
- Note como a função `obterIMC` retorna `double`, não `int` — precisamos da precisão fracionária. `setprecision(2)` (de `<iomanip>`) garante apresentação com duas casas decimais.
- A altura está representada como `int` (centímetros) por simplicidade didática e seguindo o enunciado. Em código real, preferiríamos `double` em metros, ou pelo menos uma documentação explícita das unidades.

Gráfico do IMC

O “gráfico” mencionado pelo enunciado é implementado como a tabela textual de classificação que aparece na saída (função `exibirTabelaIMC`). Em uma versão mais sofisticada com biblioteca gráfica, poderíamos plotar barras coloridas e indicar visualmente em que faixa o usuário se encontra; fora do escopo deste exercício puramente textual.

Observação de Engenharia de Software

Sobre privacidade. O enunciado do exercício menciona que a informatização de registros de saúde levanta sérias questões de privacidade. Nosso `PerfilSaude` guarda dados extremamente sensíveis (nome completo, data de nascimento, peso) em memória sem criptografia, sem controle de acesso e sem auditoria. Em uma aplicação real, esses dados teriam de obedecer a legislações como a LGPD (Lei Geral de Proteção de Dados, no Brasil), a HIPAA (*Health Insurance Portability and Accountability Act*, nos EUA) ou GDPR (Europa) — com requisitos rigorosos de criptografia, consentimento informado e direito ao esquecimento. Esses temas voltarão em capítulos futuros sobre persistência e segurança.

Boa Prática de Programação

Os `setters` desta classe não validam a maioria dos campos (nome, peso, altura). Em código de produção, validações esperadas: nome não vazio, sexo $\in \{\text{“M”}, \text{“F”}\}$ (ou outro modelo de gênero), data válida (ver Exerc. 16.15), peso e altura positivos e em faixas plausíveis. Implementar essas validações é um excelente exercício adicional para o leitor.

Encerramento

Os dois exercícios *Fazendo a Diferença* do Capítulo 16 ilustram, em escala modesta mas legítima, como a programação orientada a objetos pode contribuir para projetos de impacto social — aqui, na área de saúde e bem-estar.

A classe `FrequenciaCardiaca` encapsula a lógica de cálculo de uma fórmula da American Heart Association de maneira que o código cliente não precisa conhecer os detalhes (220 – idade, percentuais 50%–85%). Se a AHA atualizar sua orientação no futuro — digamos, ajustando a constante ou a faixa percentual —, basta editar a classe em um único lugar e todos os programas que a usam serão automaticamente atualizados.

A classe `PerfilSaude` dá um passo além ao combinar dois cálculos derivados (IMC e frequência cardíaca) em uma única abstração da pessoa-paciente. É exatamente o tipo de abstração composta

que sistemas reais de registros eletrônicos de saúde precisarão construir — centenas de vezes, em centenas de áreas clínicas.
